

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

Nombre del alumno:

ANGEL GASPAR REYES

8A

TSU EN TECNOLOGIAS DE LA
INFORMACION: DESARROLLO DE
SOFTWARE Y MULTIPLATAFORMA

TEHUACÁN, PUEBLA, 29 de Marzo de 2026

Recuperación de contraseñas accesible y segura

1. ¿Qué es un token de recuperación?

Un **token de recuperación** es un código único, aleatorio y temporal que se genera cuando un usuario solicita restablecer su contraseña.

Funciona como una “llave temporal” para verificar que el usuario realmente tiene acceso a su correo.

Características importantes:

- Es **único**
- Es **difícil de adivinar** (random, largo)
- Se usa **una sola vez**
- Tiene **tiempo de expiración**

Ejemplo:

<https://miapp.com/reset-password?token=abc123XYZ789>

2. ¿Por qué deben expirar los tokens?

Si no expiran... básicamente estás dejando la puerta abierta

Riesgos si no expiran:

- Un atacante puede usar un token viejo
- Correos filtrados pueden ser reutilizados
- Accesos no autorizados mucho tiempo después

Buenas prácticas:

- Expiración de **15 a 60 minutos**
- Invalidar token después de usarlo
- Permitir generar uno nuevo (invalidando el anterior)

3. Ataques comunes en recuperación de contraseña

Aquí es donde muchos sistemas fallan

1. Enumeración de usuarios

Cuando el sistema dice:

- “Correo no registrado”

El atacante descubre qué cuentas existen.

✓ Solución:

- Mostrar siempre:
 “Si el correo existe, recibirás instrucciones”

● 2. Fuerza bruta de tokens

Intentar adivinar tokens.

✓ Solución:

- Tokens largos (mínimo 32 caracteres)
- Uso de hashing en base de datos
- Limitar intentos

● 3. Reutilización de tokens

Un token viejo sigue funcionando.

✓ Solución:

- Expiración + invalidación tras uso

● 4. Interceptación (MITM)

Si no usas HTTPS, el token puede ser robado.

✓ Solución:

- Siempre usar **HTTPS**

● 5. Phishing

Correos falsos que imitan tu sistema.

✓ Solución:

- Correos claros
- Dominio oficial
- No incluir datos sensibles

4. Diseño de flujo accesible (UX + Seguridad)

Aquí es donde te vuelves pro: no solo seguro... también usable.

✿ Flujo completo

1. Solicitud de recuperación

Formulario:

- Campo: correo electrónico

✓ Accesibilidad:

- Etiquetas claras (label)
- Soporte para teclado
- Mensajes entendibles

✓ Seguridad:

- No revelar si el correo existe

2. Envío de token

- Se genera token
- Se guarda en DB (hasheado)
- Se envía por correo

3. Validación del token

Usuario entra al link: `/reset-password?token=XYZ`

✓ Validar:

- Existe
- No expiró
- No ha sido usado

4. Cambio de contraseña

Formulario:

- Nueva contraseña
- Confirmación

✓ Accesibilidad:

- Mostrar errores claros:
 - “La contraseña es muy corta”
- Navegación por teclado
- Compatible con lectores de pantalla

✓ Seguridad:

- Validación fuerte (mínimo 8 caracteres, etc.)
- No permitir contraseñas comunes

5. Confirmación

Mensaje:

- “Contraseña actualizada correctamente”

✓ Opcional PRO:

- Cerrar todas las sesiones activas

5. Mensajes accesibles y seguros

Aquí hay equilibrio importante 

Incorrecto:

- “El correo no existe”
- “Token inválido para usuario Juan”

Correcto:

- “Si el correo está registrado, recibirás instrucciones”
- “El enlace no es válido o ha expirado”

✓ Claros

✓ No revelan información

✓ Entendibles para cualquier usuario

6. ¿Cómo implementarlo técnicamente?

Te lo dejo aterrizado a algo que puedes programar:

Backend (ejemplo conceptual)

```
// Generar token
```

```
const token = crypto.randomBytes(32).toString("hex");
```

```
// Guardar hash
```

```
const tokenHash = hash(token);
```

```
// Guardar en DB
```

```
{
```

```
  email,
```

```
  token: tokenHash,
```

```
  expiresAt: Date.now() + 3600000 // 1 hora
```

```
}
```


Validación

```
if (!tokenExiste || expirado) {  
    return "Token inválido o expirado";  
}
```